

Configuration Développement - JobbingTrack

Guide de configuration de l'environnement de développement pour JobbingTrack v4.1.

[← Retour à la documentation](#) | [← README principal](#) |  [Navigation](#)

Vue d'ensemble

Configuration complète pour développer sur JobbingTrack avec Node.js, TypeScript, Prisma, Docker et les tests.

Prérequis

Outils système

- **Node.js** : 20 LTS
- **npm** : 8.0+ ou **yarn** : 1.22+
- **Docker** : 20.10+
- **Docker Compose** : 2.0+
- **Git** : 2.30+

IDE recommandé

- **Visual Studio Code** avec extensions :
 - ES7+ React/Redux/React-Native snippets
 - Prettier
 - ESLint
 - Docker
 - Prisma
-

Configuration rapide

1. Installation des outils

```
# Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Docker
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
sudo sh get-docker.sh

# Docker Compose
sudo apt-get install docker-compose-plugin
```

2. Clonage et installation

```
# Clonage du projet
git clone https://github.com/votre-repo/jobbingtrack.git
cd jobbingtrack

# Installation des dépendances
npm install

# Configuration de l'environnement
cp .env.example .env
nano .env
```

3. Démarrage en mode développement

```
# Services de développement
make dev

# Ou manuellement
docker-compose -f docker-compose.dev.yml up -d

# Installation des dépendances frontend
cd frontend && npm install
cd ../backend && npm install

# Démarrage du frontend en mode dev
cd frontend && npm run dev

# Démarrage des services backend
cd backend && npm run dev:services
```



Configuration détaillée

Variables d'environnement

Développement (.env)

```
# Base de données de développement
DATABASE_URL=postgresql://jobbingtrack:jobbingtrack123@localhost:5432/jobbingtr

# JWT pour le développement (moins sécurisé mais pratique)
JWT_SECRET=dev-jwt-secret-key-2025
JWT_REFRESH_SECRET=dev-refresh-secret-2025

# Services de développement
REDIS_URL=redis://localhost:6379
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8080,http://localhost:3

# Frontend
FRONTEND_URL=http://localhost:8080
NEXT_PUBLIC_API_URL=http://localhost:3000

# Email (optionnel en dev)
SMTP_HOST=localhost
SMTP_PORT=1025
SMTP_USER=
SMTP_PASS=

# Logs
LOG_LEVEL=debug
NODE_ENV=development
```

Production (.env.production)

```
DATABASE_URL=postgresql://jobbingtrack:secure_password@postgres:5432/jobbingtr
JWT_SECRET=your-production-secret-key-2025
JWT_REFRESH_SECRET=your-production-refresh-secret-2025
REDIS_URL=redis://redis:6379
ALLOWED_ORIGINS=https://yourdomain.com,https://app.yourdomain.com
FRONTEND_URL=https://yourdomain.com
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASS=your-sendgrid-api-key
LOG_LEVEL=info
NODE_ENV=production
```

Configuration Docker Compose pour le développement

docker-compose.dev.yml

```

version: '3.8'
services:
  postgres-dev:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: jobbingtrack_dev
      POSTGRES_USER: jobbingtrack
      POSTGRES_PASSWORD: jobbingtrack123
    ports:
      - "5433:5432"
    volumes:
      - postgres_dev_data:/var/lib/postgresql/data
    command: postgres -c log_statement=all -c log_destination=stderr

  redis-dev:
    image: redis:7-alpine
    ports:
      - "6380:6379"
    command: redis-server --appendonly yes

# Services backend avec hot reload
auth-service-dev:
  build:
    context: ./backend/auth-service
    dockerfile: Dockerfile.dev
  volumes:
    - ./backend/auth-service/src:/app/src
    - ./backend/auth-service/nodemon.json:/app/nodemon.json
  environment:
    NODE_ENV: development
  ports:
    - "3001:3000"

# Frontend avec hot reload
frontend-dev:
  build:
    context: ./frontend
    dockerfile: Dockerfile.dev
  volumes:
    - ./frontend/src:/app/src
    - ./frontend/public:/app/public
  ports:
    - "8080:3000"
  environment:
    NODE_ENV: development

```

Dockerfile.dev pour les services

```
FROM node:20-alpine

WORKDIR /app

# Installation des dépendances
COPY package*.json ./
RUN npm install

# Configuration nodemon pour hot reload
COPY nodemon.json ./
RUN npm install -g nodemon

# Copier le code source
COPY . .

EXPOSE 3000

# Démarrage en mode développement
CMD ["nodemon", "src/server.js"]
```



Tests et qualité

Configuration des tests

Jest (Backend)

```
// backend/jest.config.js
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  roots: ['<rootDir>/src', '<rootDir>/tests'],
  testMatch: ['**/__tests__/**/*.ts', '**/?(*.)+(spec|test).ts'],
  transform: {
    '^.+\\.ts$': 'ts-jest',
  },
  collectCoverageFrom: [
    'src/**/*.ts',
    '!src/**/*.d.ts',
    '!src/server.ts',
  ],
  coverageDirectory: 'coverage',
  coverageReporters: ['text', 'lcov', 'html'],
  setupFilesAfterEnv: ['<rootDir>/tests/setup.ts'],
}
```

```
testTimeout: 10000,
};
```

Playwright (Frontend)

```
// frontend/playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:8080',
    trace: 'on-first-retry',
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
  ],
  webServer: {
    command: 'npm run build && npm run preview',
    url: 'http://localhost:8080',
    reuseExistingServer: !process.env.CI,
  },
});
```

Commandes de test

```
# Tests backend (tous services)
make test

# Tests frontend
cd frontend && npm run test

# Tests d'intégration
make test-integration

# Tests E2E avec Playwright
make test-e2e

# Tests de performance
```

```
make test-performance
```

```
# Coverage  
make coverage
```

Linting et formatage

```
# Linting backend  
cd backend && npm run lint  
  
# Linting frontend  
cd frontend && npm run lint  
  
# Formatage automatique  
make format  
  
# Type checking  
cd frontend && npm run type-check
```



Workflow de développement

Git Hooks (Husky)

```
# Installation des hooks  
cd frontend && npm run prepare  
  
# Hooks configurés :  
# - pre-commit: linting et tests  
# - pre-push: tests complets  
# - commit-msg: format des messages
```

Développement avec Docker

```
# Démarrage en mode développement  
make dev  
  
# Services avec hot reload  
docker-compose -f docker-compose.dev.yml up  
  
# Build des services
```

```
make build-dev

# Logs en temps réel
make logs-dev
```

Développement sans Docker

```
# Base de données locale
sudo -u postgres createdb jobbingtrack_dev
sudo -u postgres psql -c "CREATE USER jobbingtrack WITH PASSWORD 'jobbingtrack'"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE jobbingtrack_dev TO jobbingtrack;"

# Démarrage des services en local
cd backend && npm run dev:local

# Frontend en local
cd frontend && npm run dev
```



Debugging

Logs de développement

```
# Logs de tous les services
make logs

# Logs d'un service spécifique
make logs SERVICE=auth-service

# Logs avec suivi
make logs-follow

# Logs du frontend
cd frontend && npm run logs
```

Outils de debugging

VS Code Debug Configuration

```
{
  "version": "0.2.0",
```

```
"configurations": [
  {
    "name": "Debug Auth Service",
    "type": "node",
    "request": "launch",
    "program": "${workspaceFolder}/backend/auth-service/src/server.js",
    "skipFiles": ["<node_internals>/**"],
    "env": {
      "NODE_ENV": "development",
      "DATABASE_URL": "postgresql://localhost:5433/jobbingtrack_dev"
    }
  },
  {
    "name": "Debug Frontend",
    "type": "node",
    "request": "launch",
    "program": "${workspaceFolder}/frontend/node_modules/.bin/next",
    "args": ["dev"],
    "cwd": "${workspaceFolder}/frontend"
  }
]
```

Postman pour les tests API

```
# Collection Postman disponible dans docs/api/
# Variables d'environnement :
# - baseUrl: http://localhost:3000
# - authToken: Bearer <token>
```

Métriques de développement

```
# Métriques Prometheus locales
curl http://localhost:9090/api/v1/query?query=up

# Health checks
curl http://localhost:3000/health
curl http://localhost:3001/health
curl http://localhost:8080/api/health
```



Performance et optimisation

Bundle Analyzer

```
# Frontend bundle analysis
cd frontend && npm run analyze

# Backend bundle analysis
cd backend && npm run bundle-analyze
```

Profiling

```
# Node.js profiling
node --prof src/server.js
node --prof-process isolate-*.log > profile.txt

# Frontend performance
cd frontend && npm run lighthouse
```



Ressources de développement

Documentation technique

- [Architecture microservices](#)
- [API Reference](#)
- [Base de données](#)
- [Tests et qualité](#)

Outils de développement

- [Makefile documentation](#)
- [Scripts d'automatisation](#)
- [Configuration CI/CD](#)

Communauté et support

- [Contributing Guide](#)
- [Code of Conduct](#)
- [Issues GitHub](#)

Version: 4.1 - Environnement de développement **Dernière mise à jour:** Octobre 2025